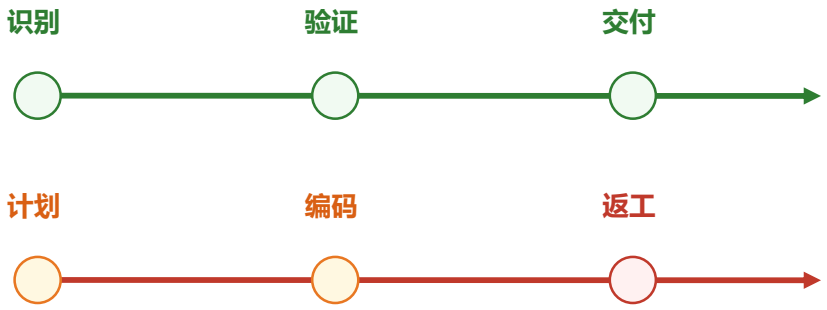
面对复杂性，开发节奏怎么选

过程模型选择与代价分析

复杂性来源不同，开发节奏不能相同



承接

上讲三把尺，今天继续用

第1次课判断复杂性，第2次课判断节奏。

复杂性四来源

- 需求变化速率
- 协作规模
- 技术组合深度
- 运行约束

FR / QA / C

- 功能需求
- 质量属性
- 约束条件

架构时间账

- 现在验证
- 未来返工
- 决策值不值

→

节奏选择

- 何时验证
- 如何交付
- 错了多贵

从复杂性诊断到过程模型选择：同一项目，不同节奏，代价不同。

案例

一个节奏选错的10周项目

不是代码写不好，是反馈来得太晚。

表面原因

- 需求变了
- 时间不够
- 测试太晚

第7周

限规则变

色与流程分

工程根因

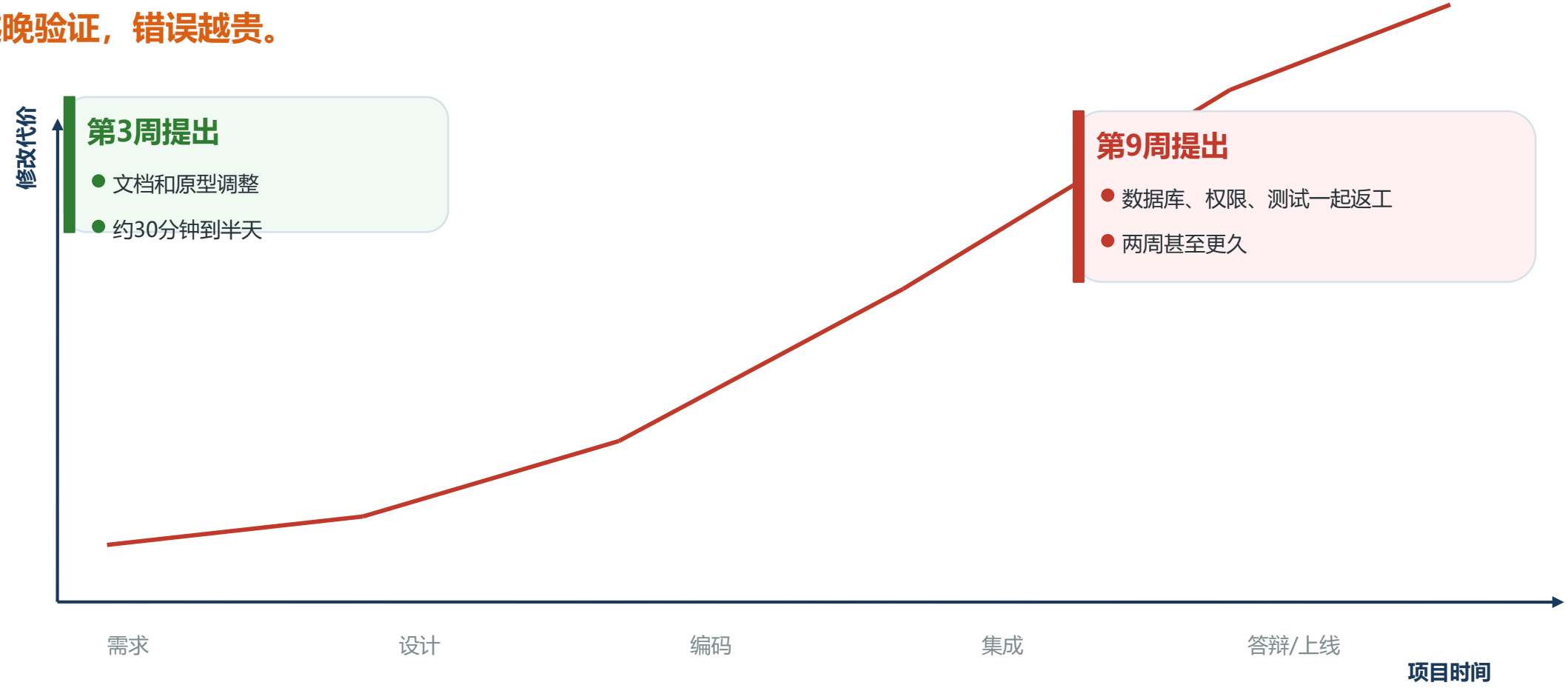
- 反馈点太晚
- 权限风险未验证
- 用瀑布节奏处理探索问题

先问复杂性，再定节奏；不能只问代码什么时候写完。

案例

错误节奏的代价曲线

越晚验证，错误越贵。



过程模型选择，本质是在安排错误暴露的时间点。

工具

本讲核心工具：四维节奏判断矩阵

选择过程模型，本质是选择反馈回路的频率和位置。

维度	高值含义	倾向节奏	错选代价
需求稳定性	变更少、合同强	瀑布可用	过度迭代导致管理成本
风险强度	技术或质量不确定	原型 / 螺旋	后期集成爆雷
交付压力	早期必须可展示	增量 / 敏捷	期末才发现不可用
协作规模	多人、多接口	强流程 / RUP裁剪	沟通路径爆炸

综合题先判断约束，再给模型；不能先喊一个模型名。

定义

什么是软件过程模型

过程模型规定的是活动顺序、角色协作、产物交付和反馈方式。

活动

需求、设计、编码、测试如何排列

角色

谁负责、谁评审、谁确认

产物

文档、原型、代码、测试报告

反馈

何时发现错误，如何回退

一句话：谁在何时产出什么，并如何校正。

问题冲击

混沌开发什么时候失控

项目超过个人控制范围后，随便写就会变成系统性风险。

1. 人数不少于3人

协作成本不再线性

2. 周期超过6周

上下文开始遗忘

3. 依赖超过5个模块

接口冲突难追踪

4. 需求来自外部用户

不是自己说了算

满足其中一条，就不能再把项目当成个人小脚本。

框架

过程模型的价值：建立反馈回路

好的过程模型不是增加手续，而是让错误更早暴露。

无反馈

- 写完才看见
- 用户最后才参与
- 测试最后才开始
- 错误集中爆发

有反馈

- 原型先验证理解
- 评审暴露偏差
- 测试前置设计
- 增量持续交付

过程模型的核心不是流程名称，而是反馈回路。

工具

开发节奏的五个判断问题

不先问条件，就不能直接选模型。

01	**需求稳不稳?**	稳定则可前置规划
02	**风险高不高?**	高风险先验证
03	**用户能否频繁反馈?**	高频反馈适合迭代
04	**是否要早交付?**	早交付倾向增量
05	**团队协作重不重?**	协作重则要流程纪律

综合题先写五问，再给模型选择和错选代价。

训练

课堂训练：给实验室预约系统打分

节奏选择要从项目约束出发，而不是从模型偏好出发。

维度	评分	依据	节奏影响
需求稳定性	中低	审批、权限、预约规则会变化	需要迭代
风险强度	中	并发冲突和越权风险	需要原型/测试前置
交付压力	高	每个里程碑要可展示	需要增量
团队协作	中	3-5人课程团队	需要轻量流程
验证机会	中高	课堂和同学可评审	反馈要频繁

学生动作：1分钟独立打分，2分钟同桌比较差异。

模型

瀑布为什么会出现

瀑布不是落后模型，它来自需求稳定、验证昂贵的历史场景。

国防合同

- 需求写进合同
- 变更流程严格
- 改动极其昂贵

硬件嵌入式

- 硬件生产后难改
- 软件需同步
- 前期设计重要

大型机软件

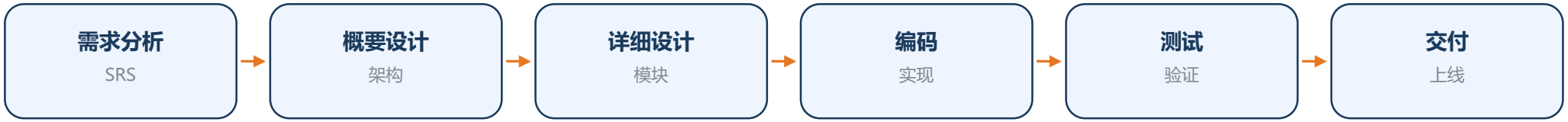
- 测试环境稀缺
- 发布机会少
- 验证成本高

瀑布本质：把不确定性集中在前期处理。

模型

瀑布模型的基本定义

瀑布是阶段式、文档驱动、顺序推进的开发节奏。



定义要点

- 阶段顺序明确
- 阶段产物明确
- 前一阶段完成后进入后一阶段
- 适合需求稳定、验证昂贵的场景

模型

真瀑布也需要反馈

刚性瀑布的风险在于切断反馈，不在于有阶段。

单向瀑布

- 需求
- 设计
- 编码
- 测试

带反馈瀑布

- 需求和设计可回看
- 设计和实现可校正
- 测试问题能追溯前阶段

反馈不是敏捷专属；任何工程过程都不能切断反馈。

对照

瀑布适用条件与失效条件

瀑布是条件苛刻的模型，条件不满足时后期爆雷。

适用条件

- 需求来自合同或法规
- 项目已有大量先例
- 用户不能频繁介入
- 验证机会少且昂贵

失效条件

- 探索型需求
- 验证只在最后
- 阶段切割太硬
- 文档与代码分叉

瀑布不是坏模型，是使用条件要求高的模型。

案例

游戏领域社交网站：瀑布为什么危险

探索型产品用瀑布，常常把“不知道”伪装成“已确定”。

未知内容

- 用户为何留下
- 什么内容有吸引力
- 关系链如何形成

瀑布风险

- 把假设写成需求
- 最后才看真实行为
- 重做产品机制

模型

原型模型的定义

原型的目的不是提前做完系统，而是尽快获得学习结果。

定义

- 快速构造可观察版本
- 用于澄清需求
- 用于验证体验或可行性

不是

- 不是低质量正式系统
- 不是跳过设计
- 不是把原型代码直接上线

价值

- 让用户看见
- 让风险提前暴露
- 让后续设计少返工

原型先回答：我们到底要做什么，能不能做成。

工具

三类原型分别验证什么

不同不确定性，要用不同原型验证。

原型类型	验证问题	实验室预约例子	输出
界面原型	用户是否看得懂	预约页面和筛选条件	可点击界面
流程原型	业务路径是否合理	提交、审批、撤销流程	流程图/泳道图
技术原型	技术能否支撑	扫码登录、并发冲突	小程序/压测脚本

先分清不确定性，才能选对原型。

案例

游戏社交网站如何用原型

用户说不清需求时，让用户看见东西比追问更有效。

假设
玩家想组队

→

原型
关系链界面

→

观察
真实试用

→

修正
调整激励

要验证的假设

- 玩家愿意添加陌生好友
- 内容流能促成互动
- 组队机制能提升留存

不应直接瀑布化

- 需求写得完整不等于真实成立
- 用户行为必须被观察
- 产品机制需要反复校正

风险

原型模型的风险

原型能降低需求风险，也可能制造架构债。

原型被当正式系统

赶进度时直接上线，留下脆弱代码

只验证界面

业务规则和性能风险仍未暴露

原型边界不清

丢弃式与演化式混淆

原型要写清楚：验证什么、是否丢弃、哪些代码能进入正式系统。

训练

课堂训练：先验证什么

先验证最不确定、最昂贵、最影响后续结构的部分。

预约冲突检测

- 规则复杂，影响数据结构
- 选择：界面/流程/技术原型
- 给出一句理由

微信扫码登录

- 第三方 OAuth 技术不确定
- 选择：界面/流程/技术原型
- 给出一句理由

审批流界面

- 用户理解和流程体验不确定
- 选择：界面/流程/技术原型
- 给出一句理由

学生动作：独立排序，说明先做哪个原型。

对照

增量与迭代的核心区别

增量是加新功能，迭代是改进已有理解。

增量开发

- 功能子集叠加
- 每次交付新能力
- 已有功能尽量不动
- 类比：盖楼

迭代开发

- 同一系统持续精化
- 已有功能可以修改
- 反馈驱动修正
- 类比：雕塑

考试高频：30秒说清“加新功能”和“改已有理解”的差别。

模型

增量开发：先交付可用子集

增量的价值在于早交付、早使用、早暴露集成问题。

V1
预约提交
能独立运行

→

V2
审批
增加处理流

→

V3
通知
消息提醒

→

V4
统计报表
管理分析

判断规则

- 每个增量都应该是可运行、可验证、可演示的能力；如果 V1 不能独立运行，就不是有效增量。

模型

迭代开发：持续修正同一能力

迭代的价值在于持续校正错误理解。

第1轮
简单规则
同一设备不重叠

→

第2轮
维护窗口
设备不可预约

→

第3轮
优先级
导师课题优先

→

第4轮
课程冲突
跨系统校验

为什么是迭代

- 一直在修改同一能力：预约冲突检测
- 已有规则会被修正或替换

考试判断

- 看是否在加新功能
- 还是在修正已有功能理解

框架

现代项目常常同时增量又迭代

一个 Sprint 可以既交付新功能，也修正已有功能。

周期	新增功能	修正已有理解	判断
Sprint 1	登录、预约提交	确认预约核心流程	增量 + 迭代
Sprint 2	审批功能	重构冲突检测	增量 + 迭代
Sprint 3	通知功能	优化权限边界	增量 + 迭代

实际项目常是组合节奏，不是单一标签。

案例

增量/迭代错选的代价

把需要迭代的问题当增量做，会把错误一层层固化。

V1
先到先得
规则写死

→

V2
导师优先
权限层改动

→

V3
设备维护
数据库改动

→

V4
课程冲突
测试用例重写

根因

- 预约规则本来就需要迭代学习，却被当作固定增量一层层往上叠；后期每次变化都会牵动数据、权限和测试。

训练

课堂训练：给定约束选节奏

模型选择要能解释“如果选错会怎样失败”。

需求稳定报表系统

- 推荐：瀑布/轻量阶段式
- 错选：频繁迭代浪费管理成本
- 写出一句理由

探索型社交产品

- 推荐：原型 + 迭代
- 错选：瀑布后期发现没人用
- 写出一句理由

实验设备预约系统

- 推荐：原型 + 增量 + 迭代
- 错选：规则变化导致返工
- 写出一句理由

答案结构：推荐节奏 + 选择理由 + 错选代价。

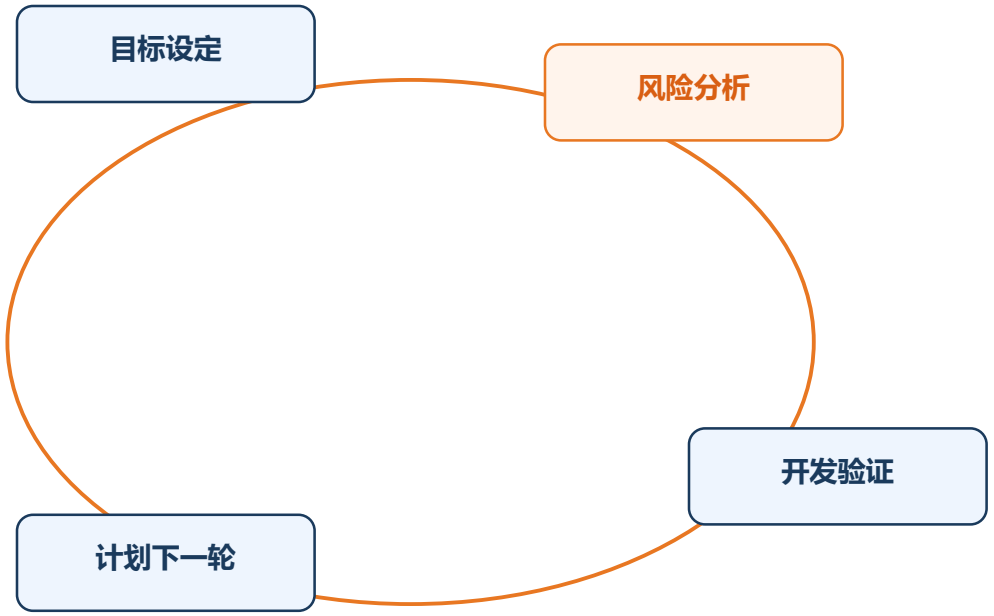
模型

螺旋模型：风险驱动的循环

螺旋模型适合风险高、投入大、不能一次赌到底的项目。

关键词

- 风险驱动
- 多轮循环
- 每轮都验证
- 不能一次赌到底



失败代价高于试验成本时，先做风险验证。

模型

螺旋模型适用场景

当失败代价高于试验成本，先做风险验证更合理。

智能硬件

- 硬件软件强耦合
- 先验证接口

复杂算法

- 效果不可提前保证
- 先验证指标

跨系统集成

- 外部接口不可控
- 先验证协议

关键性能

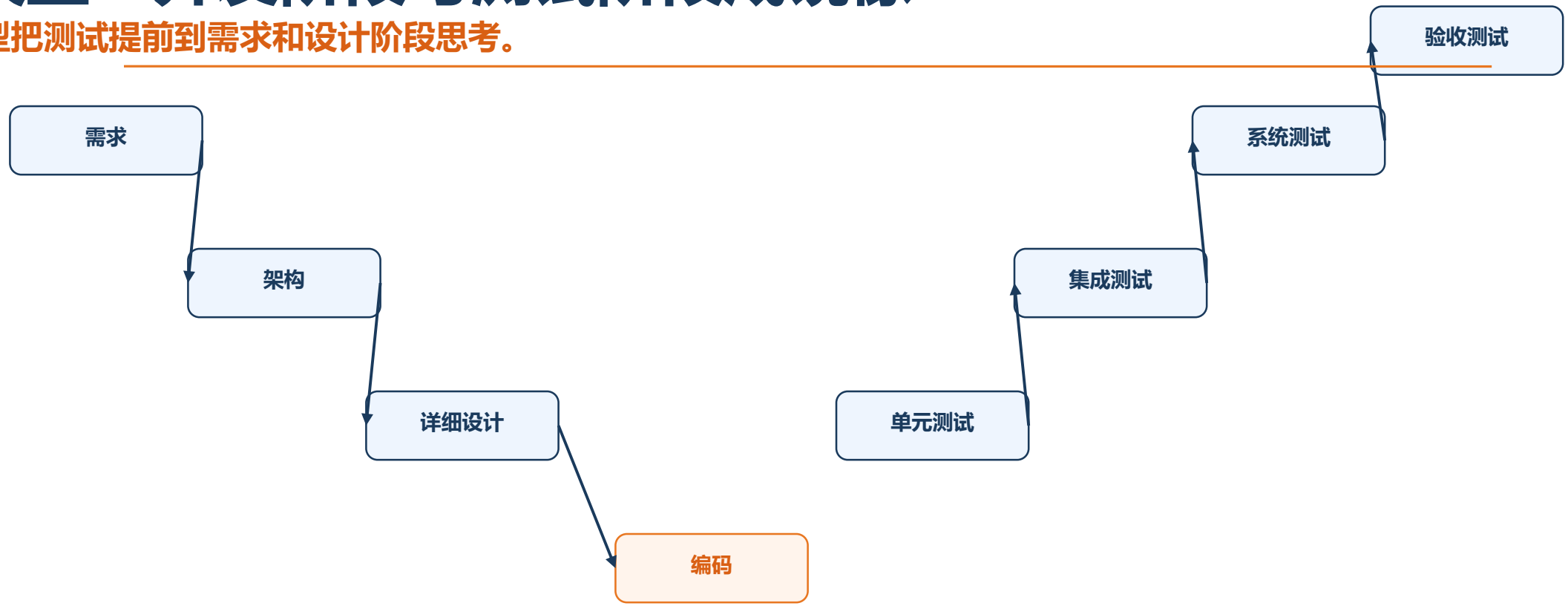
- 并发或延迟要求高
- 先做压测

问法：实验室预约系统的并发冲突是否值得先做风险验证？

模型

V模型：开发阶段与测试阶段成镜像

V模型把测试提前到需求和设计阶段思考。



需求对应验收测试，设计对应集成/系统测试

模型

V模型适用条件与错用代价

质量和合规强约束越高，测试设计越不能拖到最后。

适用

- 嵌入式和医疗
- 政务和数据安全
- 强验收标准
- 需求必须可测

错用代价

- 集成后才发现需求不可测
- 安全验收补不了
- 质量属性无法证明
- 返工范围扩大

给出 QA 时，要能说明对应测试层级。

模型

RUP：用例驱动、架构中心、迭代推进

RUP 的重点不是流程复杂，而是早期建立架构基线。

初始	细化	构造	移交
范围	架构基线	分批实现	部署交付
商业目标	高风险用例	迭代开发	培训和验收

三个关键词

- 用例驱动：从用户目标组织需求
- 架构中心：早期稳定关键结构
- 迭代推进：分批实现和验证

模型

RUP 适合什么复杂系统

系统边界多、团队多、架构风险高时，需要更强的架构纪律。

预约系统边界

- 统一身份认证
- 教务系统
- 设备台账
- 消息通知

→

架构基线

- 身份边界
- 接口协议
- 数据归属
- 错误处理

→

裁剪使用

- 3人项目不需要完整RUP
- 但要保留架构基线思想

RUP 不等于大而全流程；课程项目应裁剪使用。

训练

场景匹配训练：五个项目选模型

过程模型是场景匹配题，不是偏好题。

场景	主要复杂性	推荐节奏	错选代价
法规报送系统	需求稳定、合规强	瀑布/V模型	验收材料缺失
游戏社交产品	需求探索	原型+迭代	做完没人用
智能设备控制	硬件和安全风险	螺旋/V模型	硬件联调失败
校园预约系统	规则变化+早展示	原型+增量+迭代	后期权限返工
算法验证平台	效果不确定	原型/螺旋	投入后指标不达标

模型

敏捷为什么出现

敏捷回应的是需求变化快、反馈太晚、计划失真的现实。

大计划困境

- 做完才发现做错
- 计划越到后面越假
- 用户最后才看到
- 风险集中爆发

小批量反馈

- 短周期交付
- 频繁评审
- 持续调整优先级
- 用工作软件度量进度

敏捷解决的核心不是写代码快，而是反馈更短。

模型

敏捷不是没有文档

敏捷强调工作软件优先，但不取消必要文档。

误解

- 没有文档
- 没有计划
- 想到哪做到哪
- 口头沟通代替验收

正确理解

- 工作软件优先
- 必要文档服务协作
- 验收标准必须清楚
- 计划可调整但不能缺失

敏捷宣言中的“高于”，不是“取代”。

术语

Scrum 术语先讲清楚

听懂 Scrum，先要听懂 Backlog、Sprint、Increment 和 Done。

Product Backlog

所有待做事项的优先序清单

Sprint Backlog

本轮迭代承诺完成的事项

Sprint / Timebox

固定时长的迭代周期

Increment / Done

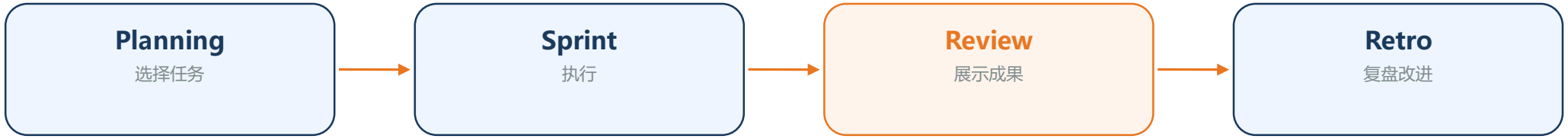
本轮完成且可验收的增量

名词解释要能映射到自己的课程项目。

模型

Scrum 活动如何组织反馈

Scrum 的核心动作是不断选择、执行、展示、复盘。



研究生团队裁剪

- 不用每天站会，但每次课前应有15分钟同步；每个里程碑必须有可展示产物。

工具

看板、Ticket、PR 与 CI

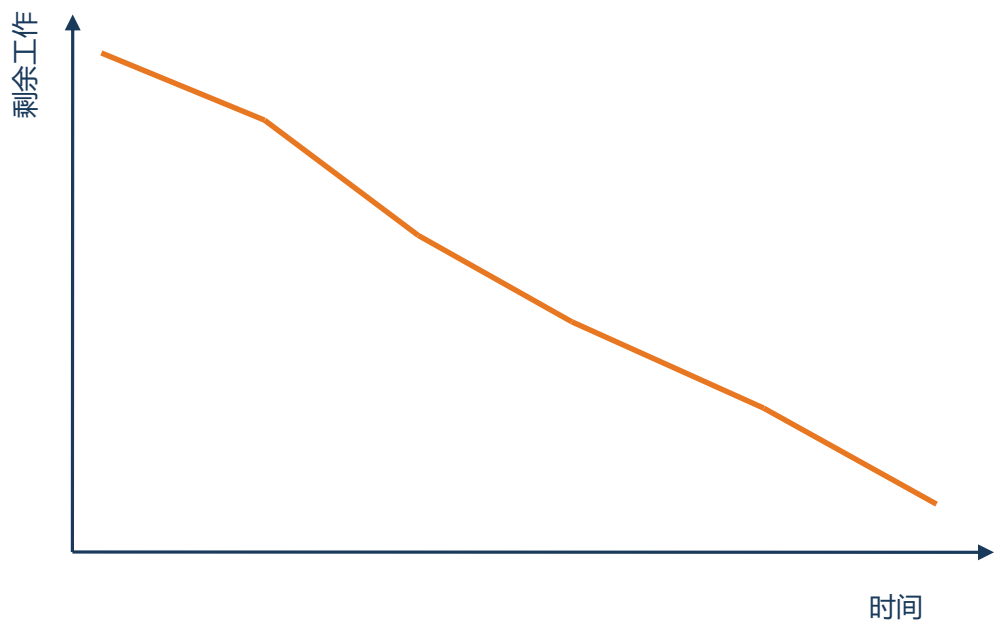
敏捷需要可视化流动和工程自动化，不是口头说快。

工具	解决什么	课程项目例子	风险
看板	任务流动可见	Todo/Doing/Done	任务堆积不可见
Ticket	任务边界清楚	完成预约提交功能	口头任务遗忘
PR	代码评审	合并前同伴检查	个人代码破坏整体
CI	自动测试	提交后跑测试	集成风险后置

工具

燃尽图与估算扑克

进度管理不是问“做完了吗”，而是持续暴露剩余工作和估算不确定性。



燃尽图

- 如果曲线连续三天不下降，说明任务卡住或估算失真

估算扑克

- 通过团队估算差异，提前发现理解不一致

风险

敏捷失效条件

没有清晰 Backlog、验收标准和测试纪律，敏捷会退化成混沌开发。

任务太大

一个 Ticket 做一周还说不清

无验收标准

Done 只是自我感觉

没有评审

用户和同伴看不到成果

没有自动测试

越改越不敢改

敏捷不是随意开发，而是更高频的纪律化反馈。

案例

实验室预约系统的节奏诊断

统一案例不适合纯瀑布，也不适合无纪律敏捷。

维度	评分	依据	节奏结论
需求稳定性	中低	规则和权限会变化	需要迭代
技术风险	中	并发冲突、扫码登录	先做原型
交付压力	高	每阶段要可展示	增量交付
团队规模	中	3-5人协作	轻量Scrum
验证机会	中高	课堂和用户可反馈	短反馈回路

框架

推荐组合：原型 + 增量 + 迭代 + 测试前置

真实项目常用组合节奏，而不是单一模型标签。

流程/技术原型
验证不确定

→

预约核心增量
先可运行

→

审批规则迭代
持续修正

→

测试前置
保护质量

组合理由

- 原型解决不知道，增量解决早交付，迭代解决规则变化，测试前置解决质量属性。

案例

10周计划样例

计划不是写满任务，而是安排验证点。

1-2周	3-4周	5-6周	7-8周	9-10周
原型	**预约核心**	**审批冲突**	**性能权限**	**演示修正**
流程与技术验证	可提交可查询	规则迭代	测试前置	答辩版本

每两周都要有可展示产物

- 不是“还在做”，而是“这一轮验证了什么，交付了什么，下一轮修正什么”。

工具

过程模型选择决策卡

选择模型要留下理由，方便后续复盘和答辩。

字段	填写要求
项目约束	工期、团队、用户、部署、验收限制
主要复杂性	需求变化、风险、协作、运行约束
推荐节奏	模型或组合模型
关键验证点	每个阶段要验证什么
错选代价	如果不用这个节奏，会在哪里失败
备选方案	为什么不用另一种模型

训练

课堂快速测验

能解释错选代价，才算真正理解过程模型。

政务合规系统

模型？理由？错选代价？

游戏社交产品

模型？理由？错选代价？

智能硬件控制

模型？理由？错选代价？

校园预约系统

模型？理由？错选代价？

答案格式：推荐模型 + 一句理由 + 一个错选代价。

任务

课后任务与收束

把自己的项目节奏选清楚，是后续需求和架构工作的前提。

任务1	给自己的项目做四维评分
任务2	填写过程模型选择决策卡
任务3	设计两周迭代计划
任务4	写清楚错选代价

下次课抽查：为什么不用另一种过程模型？

Questions?